Rainier Ordinario - 301631145
Nelson Dang - 301578533
Derek Kang - 301618005
Kenneth Pamintuan - 301582028

CMPT 276 - Phase 3 Report

Introduction

This report covers the testing phase of our game project. We wrote unit and integration tests for the game's main components using JUnit 5, and automated everything through Maven. Testing helped us verify that individual features work correctly on their own and that different parts of the game work correctly together. It also helped us find and fix several bugs in the production code.

Unit Tests

We identified the following components as the most important to unit test. Each test class focuses on one class or feature, uses fresh objects before each test, and includes both passing and failing scenarios.

Entity: Entity is the base class for all moving characters (Player, Cop, CorruptibleCop). Since it is abstract, it is tested through the Player subclass. We tested that moving in all four directions correctly updates the character's position, that moving into a wall is rejected, and that moving outside the board boundaries is rejected.

GameEngine (27 tests): GameEngine is the main class that runs the game. It handles starting levels, updating the game each tick, and checking win and loss conditions. We tested that starting the game places the player at the correct position with 100 health and a score of 0, and that calling startGame() a second time fully resets all states. We tested that each tick advances the clock by one, that ticks stop after the game ends, and that the player loses when 300 ticks are reached. We also tested that the player loses when health hits 0 or time runs out, that reaching the exit with enough coins advances the level, that wall collisions correctly block movement, and that the coin system correctly tracks collected coins, updates the score, and resets on level transition.

Clock: We tested all public methods of Clock, including default and parameterized construction, that updateTime() increments by exactly one per call, and that subtractTime() correctly clamps to zero and ignores negative inputs.

ScoreManager: We tested score accumulation, penalties, coin collection, the coin threshold gate, and cross-level resets. Testing revealed an intentional asymmetry in the design: addPoints() ignores negative inputs, while subtractPoints() allows the score to go negative. This means trap and cop penalties can bring the score below zero, which is intentional game behaviour.

BonusRewards: We tested the BonusRewards class after refactoring it during the testing phase (see Findings). Tests verify that picking up a BonusRewards item correctly heals the player by 20 HP.

Board, Cell, and Trap: We tested the full lifecycle of a Cell, including wall and floor construction, isBlocked(), coordinate getters, and the item set/get/remove cycle. For Board, we tested getCell() boundary conditions, dimension getters across all four levels, border wall validation, and the collectItemAt() lifecycle. For Trap, we tested the applyPenalty() method with standard values, zero, negative values, and large values.

Factory Classes (CellBlockFactory, CafeteriaFactory, GeneralTimeFactory, OutdoorRecFactory): Each factory class was tested by asserting every field returned by createConfig(), including level number, name, start and exit coordinates, required coins, cop spawn positions, corruptible cop settings, and additional coin spawn locations.

Player: We tested inventory initialization, picking up items from a grid cell, picking up nothing when a cell is empty, and that player health updates correctly.

Cop: We tested that the cop correctly identifies and chases the player when nearby, that the cop moves properly, that the cop deals damage on contact, and that repeated damage is applied correctly. We also verified that the cop returns to patrolling when the player is no longer in range.

CorruptibleCop: We tested that the corruptible cop correctly gives the player a key when bribed with the right item, that bribery fails when the player does not have the correct item, and that the bribe item is removed from the player's inventory after a successful bribe.

Integration Tests

Integration tests check that multiple components work correctly together.

GameEngineIntegrationTest.java (11 tests)

- The Level 1 board loads at the correct size (47×21) from the level file, and both the player start tile and exit tile are valid walkable floor tiles.
- When the player walks over a coin, the coin count increases by 1, the score increases, and the coin is removed from the board. Walking over food heals the player by 20 HP, capped at 100.
- When the player is on the same tile as a cop and updateTick() is called, the player takes damage. Repeated cop contact reduces health to 0, ends the game as a loss, and stops the game loop.
- When the player collects enough coins and reaches the exit, the game advances to Level 2, the board updates to the correct size (53×27), the coin

counter resets to 0, the game continues running, and a score bonus is applied.
- When the player steps on a trap, both their health and score decrease.

BoardIntegrationTest.java
These tests load real level files and verify correctness across component boundaries. We confirmed that spawn and exit coordinates are walkable floor tiles on all four levels, that the exit is reachable from the spawn via pathfinding (BFS) on all four levels, that all trap and item tiles are reachable from spawn, that cop spawns land on walkable tiles, that trap penalties flow correctly through the Board → Cell → Trap chain, and that collectItemAt() correctly removes items from the live board.

Clock and ScoreManager Independence
Integration tests confirmed that Clock and ScoreManager are fully independent and time manipulation has no effect on the score.

Test Automation
All tests use JUnit 5 and are located under src/test/java/.

Test Quality
Each test method checks one specific thing and has a descriptive name that makes the purpose clear without reading the test body. We used @BeforeEach to set up fresh object instances before every test so that no test can influence the result of another. When a test requires a specific precondition — such as the player having low health, or a coin already placed on a cell — that precondition is set explicitly and confirmed with an assertion before the action under test. All assertEquals and assertTrue calls include failure messages so that when a test fails, the reason is immediately clear. We made sure to include both positive cases (things that should work) and negative cases (things that should be rejected), because testing both is the only way to confirm that methods behave correctly in all situations.

Code Coverage
Coverage was measured using JaCoCo after running mvn test.

escape

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
GameUI.GamePanel		0%		0%	37	37	113	113	11	11	1	1
GameUI		0%		0%	19	19	105	105	12	12	1	1
GameEngine		61%		46%	46	92	75	252	4	33	0	1
Cop		60%		39%	22	35	28	63	1	7	0	1
GameUI.new AbstractAction() {...}		0%		0%	3	3	5	5	2	2	1	1
LevelFactory		48%		40%	3	6	3	7	0	2	0	1
LevelConfig		89%		50%	2	13	0	23	0	11	0	1
App		0%		n/a	2	2	3	3	2	2	1	1
Board		98%		100%	0	21	2	49	0	6	0	1
CorruptibleCop		98%		80%	2	7	1	21	0	2	0	1
Cell		98%		66%	2	14	0	20	0	11	0	1
Entity		94%		n/a	1	4	1	8	1	4	0	1
Player		100%		90%	1	12	0	31	0	7	0	1
ScoreManager		100%		100%	0	13	0	26	0	10	0	1
OutdoorRecFactory		100%		n/a	0	2	0	4	0	2	0	1
GeneralTimeFactory		100%		n/a	0	2	0	4	0	2	0	1
CafeteriaFactory		100%		n/a	0	2	0	4	0	2	0	1
Clock		100%		100%	0	6	0	12	0	5	0	1
CellBlockFactory		100%		n/a	0	2	0	4	0	2	0	1
Item		100%		n/a	0	4	0	8	0	4	0	1
RegularItem		100%		100%	0	3	0	6	0	2	0	1
BonusRewards		100%		100%	0	3	0	6	0	2	0	1
ShovelPart		100%		n/a	0	2	0	4	0	2	0	1
Trap		100%		n/a	0	2	0	4	0	2	0	1
Enemy		100%		n/a	0	1	0	2	0	1	0	1
Total	3,196 of 5,113	37%	174 of 317	45%	140	307	335	783	33	146	4	25

GameEngine: The uncovered lines involve paths that only trigger after completing all four levels and specifically the final win condition requiring all three shovel parts and the endGame(true) path. Simulating four full levels in a unit test would make tests tightly dependent on specific map file layouts, making them fragile to level design changes. All core logic such as loss conditions, level transitions, item collection, cop damage, and trap handling is fully covered.

Cell: The lower branch coverage in Cell is because setWall() contains a condition that only sets the wall if the cell is not already blocked. The tests only call setWall() on an unblocked cell, so the branch where the cell is already a wall and nothing happens is never exercised.

GameUI and App: GameUI uses Java Swing, which throws a HeadlessException in the headless environment that Maven uses to run tests. Visual rendering is also outside the scope of unit testing and was verified manually through gameplay. App.main() has the same issue since it only launches the UI.

Cop (lower coverage): Some chase and patrol branches require specific multi-tick sequences and board layouts to trigger in a controlled way. The core behaviours such as detecting the player, dealing damage, and patrolling are all tested.

LevelFactory: LevelFactory contains a dispatch method that routes to the correct factory subclass based on level number. The untested branches correspond to invalid level numbers that never occur during normal gameplay. All four individual factory subclasses are tested at 100%.

Findings

Bug: Player could walk through walls When testing Player movement, a test that verified the player cannot walk into a wall was failing. Tracing the code revealed that the movement logic only checked whether the target grid cell was null and it never checked whether the cell was actually a wall. The condition was updated to include a proper wall check, and the test then passed.

Bug: ConcurrentModificationException in CorruptibleCop Tests for the bribery mechanic kept failing with a ConcurrentModificationException. The code was trying to remove the bribe item from the player's inventory while still inside the loop iterating over that inventory, which caused the program to crash. The fix was to add a break so the loop stops once the item is found, then use an iterator to safely remove it afterward.

Refactor: BonusRewards class redesigned Testing revealed that BonusRewards was designed to award multiplied coins to the player, but this did not match how food items were actually handled in the game engine. The class was refactored to heal the player by 20 HP on pickup instead, consistent with the existing food handling logic. This required updating the constructor to accept a Player instead of a ScoreManager, and updating the relevant tests accordingly.

Bug: Cop spawn overlapping player start position Integration tests for CellBlockFactory revealed that one cop spawn coordinate overlapped with the player's starting position. This was flagged and corrected in the production code.

Side effect in a query method During testing, it was noticed that Player.viewItems() contained a System.out.println() call inside a getter method, which printed to the console every time the player's inventory was accessed, including during test runs. A side effect inside a query method is a design issue and was flagged for cleanup.

Map validation confirmed The integration tests confirmed that all trap and item tiles on all four maps are reachable from the player spawn point, validating the map design across all levels.

Conclusion

We wrote a total of 262 tests across unit and integration test classes, covering the game's core logic, world and grid system, character behaviours, and persistence. Testing directly led to fixing four bugs and one refactor that improved the design of BonusRewards. Coverage is high across all logic and model classes. The main untested areas are the Swing-based UI, which cannot run in a headless test environment, and the final four-level win path in GameEngine, which was intentionally left out to avoid fragile map-dependent tests. Overall, the testing phase gave us strong confidence in the correctness of the game's core systems.